FLOW ASIA | HR, PAYROLL & MIS SYSTEM

Module Branch & First Pull Request Guide

Personal setup, creating branches for your module, and the first test PR

For	All team members and module leads (every module and team block)
Companion to	GitHub Setup, Workflow & Staging Guide, v1.0 (October 8, 2026), Sections 3 and 4
Version	1.0 | October 11, 2026

In this guide

Part	What it covers	Who
1. About this guide	What happens after the GitHub organization and repository exist	Everyone
2. Personal setup	Install Git, set your identity, create an SSH key, clone the repo, switch to staging	Every member
3. How branches work here	Branch model, naming rules, branch types, good and bad names	Every member
4. Create your module branch	Step-by-step, with a worked example, from fresh staging to merged PR	Every member
5. The first test PR	Prove that access, review routing and checks work before real development	Every team
6. Module reference tables	Team names, folders, reviewers and example branch names for all blocks	Everyone
7. Troubleshooting	Common errors at this stage and the fix	Everyone
8. For the PM and #26	Pre-flight checklist, branch protection decision, rollout tracker	PM, #26

1. About this guide

The PM and the Technical Lead (#26) create the GitHub organization, the teams and the repository *flow-asia-hrms*. This guide covers what every member does next: prepare your computer, create branches inside your own module, and prove the setup works with a first test Pull Request (PR) **before** real development starts.

It repeats the commands from the main guide (Sections 3 and 4) and adds module-specific detail. If this guide and the main guide ever disagree, ask #26.

The two steps

Step	Who	What you do	You are done when
A. Personal setup	Every member	Install Git, set your name and email, create an SSH key, clone the repo, switch to staging	ssh -T git@github.com greets you by username, and git branch shows staging selected
B. First test PR	Every team (best: every member)	Create a docs/mXX-hello branch, open a PR into staging, get it reviewed and merged	Your hello file is on staging after the merge

> **Dates**
> The checklist in the main guide targets **Oct 9** for personal setup and **Oct 9-10** for the first test PR. If those dates have passed, do both now, before any feature work. The PM confirms the current dates.

Before you start

[ ] You have a GitHub account, and you will use the **same email** in Git as on GitHub.

- [ ] You accepted the organization invitation and turned on **two-factor authentication**.
- [ ] You can see your module team under the organization's Teams. If not, tell your module lead.
- [ ] The PM or #26 confirmed that the repository *flow-asia-hrms* exists with **main** and **staging**.
- [ ] You know your module code (for example m03), your lead's number, and your module folder (Part 6).

2. Personal setup on your computer

Do this once. If a step fails, ask your module lead and do not skip ahead.

STEP 1 Install Git

- Download from https://git-scm.com/downloads. On Windows, keep the default options.
- Check the installation: `git --version` must print a version number.

STEP 2 Tell Git who you are

Use the **same email** as your GitHub account, so your commits are linked to you. Run the line for your system only where the guide says so.

```
git config --global user.name "Your Full Name"
git config --global user.email "your.email@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --global core.autocrlf input     # macOS / Linux ONLY
git config --global core.autocrlf true      # Windows ONLY (use this one instead)
git config --global --list                  # check your settings
```

STEP 3 Connect to GitHub with an SSH key

An SSH key lets your computer prove who you are to GitHub without typing a password every time.

```
ssh-keygen -t ed25519 -C "your.email@example.com"   # press Enter for every question
eval "$(ssh-agent -s)"                              # macOS / Linux / Git Bash
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub                           # show the PUBLIC key, copy the whole line
# Windows PowerShell:  Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub
```

- On GitHub: your photo > **Settings** > **SSH and GPG keys** > **New SSH key** > paste > **Add SSH key**.
- Test it: `ssh -T git@github.com` (type yes the first time). Expected: *Hi <username>! You've successfully authenticated...*
- Never share your private key (the file without .pub) or any token with anyone.

> **If SSH is blocked on your network**
> Use HTTPS with a Personal Access Token instead: GitHub > Settings > Developer settings > Personal access tokens. Use the https://github.com/ORG/flow-asia-hrms.git address when you clone, and the token as the password.

STEP 4 Clone the project and switch to staging

Replace **ORG** with your organization name (the main guide uses *flowasia-dev* as an example). The easiest way to get the exact address: open the repository on GitHub, click the green **Code** button, choose **SSH**, and copy it.

```
cd ~/projects                                  # any folder you like
git clone git@github.com:ORG/flow-asia-hrms.git
cd flow-asia-hrms
git switch staging                             # you always start from staging
git branch -a                                  # list branches
```

In the list, **staging** must have the * mark, and *remotes/origin/main* and *remotes/origin/staging* must be shown.

STEP 5 Run the project locally (when #26 gives you the commands)

Ask #26 for the exact commands. The typical Laravel + MySQL steps look like this:

```
cp .env.example .env        # then fill in YOUR local database details
composer install
php artisan key:generate
php artisan migrate --seed  # creates tables and sample data on YOUR computer
php artisan serve           # open http://127.0.0.1:8000
```

Your .env file stays on your computer

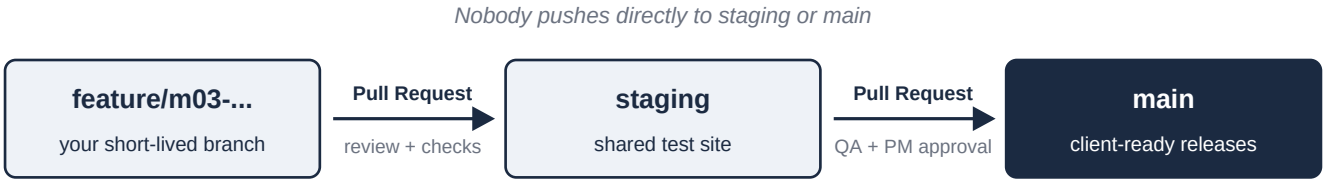
It is ignored by Git and must never be pushed. Never commit passwords, API keys, selfie photos, QR secrets or real employee or payroll data.

You are ready when

- [ ] `git --version` prints a version.
- [ ] `ssh -T git@github.com` greets you by your GitHub username.
- [ ] The project folder exists on your computer and `git branch` shows *** staging**.
- [ ] If #26 gave you the run commands: the app opens at http://127.0.0.1:8000.

3. How branches work in this project

All teams work in one repository with two long-lived branches, **main** and **staging**. Every person works on a short-lived branch of their own. Your work reaches the shared test site (staging) only through a Pull Request, and reaches main only from staging after QA and PM approval.



Branch	Purpose	Who can change it	How code gets in
main	Stable, client-ready releases. Tagged at checkpoints (v0.1.0, v0.2.0, v1.0.0).	Nobody directly	PR from staging only. Approved by #26 and the PM.
staging	Shared integration branch. Every merge is deployed to the staging test site. Default branch.	Nobody directly	PR from a feature branch. Approved by the module lead (and #26 for shared files). Tests must pass.
feature/*, fix/*, docs/*, test/*, chore/*	One branch per task. Lives a few days at most.	The branch owner	Created from **staging**, deleted after merge.

Naming rules

Item	Rule	Example

Branch	type/module-short-description. Lowercase, words separated by hyphens.	`feature/m03-qr-generation`
		`fix/m07-late-deduction`
		`docs/s3-user-manual`

Commit message	type(module): what you did. Present tense, under 70 characters.	`feat(m03): add selfie capture` `fix(m05): correct leave balance`
PR title	Commit style plus the Feature Tracker ID.	`feat(m01): audit entry for selfie rejection [F016]`
One branch = one task	One feature (or a small group of related ones). Merge small and often.	Do not keep a branch open for 2 weeks.

Branch types

Type	Use it for	Example
`feature`	New work from the Feature Tracker	`feature/m05-leave-filing-form`
`fix`	Bug fixes, including bugs QA reports as Issues. Also use it right away if you broke staging.	`fix/m07-late-deduction`
`docs`	Documentation: module write-ups, manuals, the first test PR	`docs/s3-user-manual`
`test`	Tests and test data (QA)	`test/qa-payroll-sample-data`
`chore`	Setup and housekeeping that is not a feature	`chore/s1-update-gitignore`

Good and bad branch names

Not this	Why	Use this
`my-branch`	No type and no module code	`feature/m03-qr-generation`
`Feature/M03_QR`	Capital letters and underscores	`feature/m03-qr-generation`
`feature/qr`	No module code, reviewers cannot tell whose work it is	`feature/m03-qr-generation`
`feature/m03-all-attendance`	Too big. One branch = one task.	A separate branch for each feature ID
`staging2` or `main-fix`	Looks like a long-lived branch	`fix/m03-qr-expiry`

4. Create your module branch, step by step

Every task in the Feature Tracker (F### ID) gets its own branch. The example below uses module **M3 Attendance**, member **#06**, feature **F033 (QR code generation)**. Replace the module code, folder and branch name with your own.

STEP 1 Open a terminal in the project folder and check it is clean

```
cd ~/projects/flow-asia-hrms
git status
```

You want *nothing to commit, working tree clean*. If you have unsaved work from before, put it aside with `git stash push -m "note"` or ask your lead.

STEP 2 Update staging

```
git switch staging
git pull origin staging       # get everyone's latest merged work
```

A new branch starts from the branch you are standing on. If staging is old, your branch starts from old code and you will get conflicts later. **Always do this step first, every time.**

STEP 3 Choose the branch name

- Find your task and its **F### ID** in the Feature Tracker. Section 6 of the main guide lists a suggested branch name for every feature, per member.
- Follow the pattern `type/module-short-description`. For new feature work the type is `feature`, for a bug it is `fix`.
- The suggested names in the main guide are sometimes cut short. You may use a shorter, clear name as long as it keeps the type and module code.

STEP 4 Create the branch and switch to it

```
git switch -c feature/m03-qr-code-generation-30-seconds
```

`-c` means create. Your computer now works on the new branch. Nothing is on GitHub yet.

STEP 5 Confirm you are on the right branch

```
git branch                    # the branch with * is the one you are on
git status                    # first line: On branch feature/m03-qr-code-generation-30-seconds
```

Check this before you edit anything. Work done on staging by mistake is the most common problem (see Part 7).

STEP 6 Work only inside your own module folder

- M3 example: `modules/m03-attendance/`. #06, #07 and #08 each use their own sub-folder (`/punch`, `/compute`, `/review`).
- Need to change another module's files, or a shared place (`database/`, `app/`, `.github/`)? Tell the owner first. Every migration PR needs approval from #26.
- Migration files carry a module prefix and a timestamp, for example `2026_10_09_100000_m03_create_punches_table.php`.

STEP 7 Commit in small steps

```
git status                                      # what changed?
git add modules/m03-attendance/qr/              # stage only YOUR files
git diff --staged                               # read what you are about to save
git commit -m "feat(m03): generate one-time QR code"
git log --oneline -5                            # last 5 commits
```

- Commit at least once per hour of real work. Small commits are easy to review and easy to undo.
- Avoid `git add .` unless you checked `git status` first. Never add `.env`, uploads, selfies or real employee or payroll data.

STEP 8 Push your branch (at noon and at the end of the day)

```
git push -u origin feature/m03-qr-code-generation-30-seconds   # first push of a new branch
git push                                                       # every push after that
```

After the first push your branch exists on GitHub. Git usually prints a link you can open to start the PR.

STEP 9 Open a Pull Request into staging

- On GitHub: your repository > **Pull requests** > **New pull request**. Set **base: staging** and **compare: your branch**.
- Fill in the PR template: what it does, the Feature ID, the module, how to test, and the checklist.
- Title: commit style plus the ID, for example `feat(m03): one-time QR generation [F033]`.
- In a busy module such as M3, open a **draft PR** early (Convert to draft) so reviewers can follow the work.

```
gh auth login                                 # one time
gh pr create --base staging --head feature/m03-qr-code-generation-30-seconds \
  --title "feat(m03): one-time QR generation [F033]" \
  --body "Adds a QR code that lasts 30 seconds and works once."
gh pr status                                  # see your PRs and checks
```

Before opening the PR, merge the latest staging into your branch (Step 10) so it is not out of date.

STEP 10 Continue the next day and keep up to date

```
git switch feature/m03-qr-code-generation-30-seconds
git fetch origin
git merge origin/staging       # bring in everyone's new merged work
```

If Git reports a CONFLICT, open the file, keep the correct lines, delete the marker lines (`<<<<<<<`, `=======`, `>>>>>>>`), then `git add <file>` and `git commit`. If the file belongs to another module, or is a migration, stop and ask its owner before choosing lines.

STEP 11 After your PR is merged

```
git switch staging
git pull origin staging
git branch -d feature/m03-qr-code-generation-30-seconds   # delete your local copy
git fetch --prune                                         # clean the list of remote branches
```

Then start your next task from Step 2. Each task is a new branch from fresh staging.

> **Starting a second task while the first PR waits for review**
> Go back to staging first (Step 2) and create the new branch from there. Do not branch from your other feature branch. If the second task really needs code from the first, ask your lead how to order the work.

Module-specific rules that affect your branches

These come from Section 6 of the main guide. Check the block for your module there for the full list.

Module	Rule to remember
S1 (#26, #27)	Prefix *feature/s1-*. All migration PRs from any module need #26's approval. Own the protection rules, CODEOWNERS and the two workflow files.
M1 (#01, #02)	Merge login, roles and settings first and announce it. Other modules ask #01 for new permissions through an Issue, they do not edit M1 files.
M2 (#03-#05)	Merge the core profile, departments, holiday calendar and schedule templates first. Reference photos and 201 documents never go in Git.
M3 (#06-#08)	Split into small PRs per feature and per member. Open a draft PR early. Selfies, QR secrets and signing keys are never committed.
M7 (#16-#18)	Merge in this order: period setup, earnings, deductions, recompute/lock, payslips. Merge origin/staging into your branch **every day**.
M9 (#21), M10 (#22)	After Oct 15 (#21) and Oct 19 (#22) these members also work on M12 and M11. Name those branches *feature/m12-...* and *feature/m11-...*.
M11 (#22-#24)	M11 only reads data from other modules. Stretch ML items are merged last.
M12 (#25, #21)	Merge the notification call first. The progress report form must be on staging by Oct 15.
S2 QA (#28, #29)	Test on the staging site only. The rules say branches start with *test/*, the branch plan uses *feature/qa-*. Let #28 decide one style and use it for everyone.
S3 Docs (#30-#32)	The rules say branches start with *docs/*, the branch plan uses *feature/docs-*. Let #30 decide one style. Large files stay on the shared drive.

Branch quick card

```
# every morning
git switch staging
git pull origin staging
# new task
git switch -c feature/m03-your-task
# save work
git status
git add <your-files>
git commit -m "feat(m03): what you did"
# push (first time, then after)
git push -u origin feature/m03-your-task
git push
# next day, same task
git switch feature/m03-your-task
git fetch origin
git merge origin/staging
# after the PR is merged
git switch staging
git pull origin staging
git branch -d feature/m03-your-task
git fetch --prune
```

5. The first test Pull Request

Before anyone starts real work, each team creates a small test branch, opens a PR into staging and merges it. A tiny change is enough. The goal is to prove that the whole path works.

What the test proves

Check	How you see it works
Access	You can push your branch. If GitHub says permission denied, your team or access is not set up.
Review routing	Your module lead appears as a requested reviewer on the PR (from CODEOWNERS).
Checks	The *test* check runs on the PR and turns green.
Merge rules	You cannot merge before approval and a green check. After approval you can Squash and merge.
Branch clean-up	The branch is deleted after the merge (GitHub does this automatically when enabled).

Who does it

The checklist asks for at least **one test PR per team**. It is better if **every member** does one, because that proves each person's computer, SSH key and access. To avoid two people editing the same file, put your **member number** in the branch and file name (for example *docs/m03-hello-06* and *HELLO-06.md*).

STEP 1 Create the test branch from fresh staging

```
git switch staging
git pull origin staging
git switch -c docs/m03-hello-06        # your module code and member number
```

STEP 2 Add a hello file inside your own module folder

Create a text file called **HELLO-<your number>.md** in your module folder (list in Part 6) with one line of text. Use any text editor, or one of these commands:

```
# macOS / Linux / Git Bash
echo "Hello from member 06, M3 Attendance team" > modules/m03-attendance/HELLO-06.md
# Windows PowerShell
Set-Content -Path modules/m03-attendance/HELLO-06.md -Value "Hello from member 06, M3 Attendance team"
```

STEP 3 Check, stage and commit

```
git status
git add modules/m03-attendance/HELLO-06.md
git diff --staged
git commit -m "docs(m03): add hello file for first test PR"
```

STEP 4 Push the branch

```
git push -u origin docs/m03-hello-06
```

STEP 5 Open the PR into staging

- On GitHub: **Pull requests** > **New pull request**. Base: **staging**. Compare: your test branch.
- Title: `docs(m03): hello test PR`. The template opens automatically. Use the text below.

```
## What does this PR do?
First test PR. Adds a hello file to prove my access to the M3 folder.

## Feature ID(s) from the Feature Tracker
None (test PR)

## Module
M03

## How to test
1. Open modules/m03-attendance/HELLO-06.md
2. Check that it contains one line of text.
```

Tick the checklist boxes that apply. Or use the command line:

```
gh pr create --base staging --head docs/m03-hello-06 \
  --title "docs(m03): hello test PR" \
  --body "First test PR for the M3 team. No feature ID."
```

STEP 6 Watch the checks and the reviewer

- Scroll to the bottom of the PR. The **test** check should turn green. If it is red, click Details, read the red lines, and tell #26 if the cause is not your file.
- On the right side, **Reviewers** must show your module lead. If it is empty, tell #26: the team or the CODEOWNERS file needs fixing.

STEP 7 Review and merge

- Your module lead reviews and approves (answer time: 4 working hours). Reviewers check: does it do what the PR says, does it stay in its module, is there a secret or real data, does the app still run.
- When the PR is approved and green, the **author** clicks **Squash and merge**, then **Delete branch**.
- A lead's own test PR cannot be approved by the lead. Another member of the team, a second lead or #26 approves it.
- If the reviewer asks for changes: fix on the same branch, commit, push. The PR updates by itself.

STEP 8 Clean up and tell your lead

```
git switch staging
git pull origin staging
git branch -d docs/m03-hello-06
git fetch --prune
```

Your hello file is now in your module folder on staging. Tell your lead in the team chat that your test PR is merged. Leads confirm to the PM and #26 when their whole team is done (tracker in Part 8). The hello files can stay until the first real files arrive, or the lead can remove them later in a normal *chore/* PR.

Deploy to staging
A merge into staging starts the *Deploy to staging* workflow, but only after #26 has added that workflow and the staging server exists. Until then a merge only updates the branch on GitHub. Ask #26 whether the deploy is live before you check the staging site.

6. Module reference tables

Teams, folders and reviewers

Block	GitHub team	Folder you own	Lead (reviewer)
M1 Admin, Login & Security	m01-admin	modules/m01-admin-security/	#01
M2 Employee Records & Org Setup	m02-records	modules/m02-employee-records/	#03
M3 Attendance, Selfie, QR & Review	m03-attendance	modules/m03-attendance/	#06
M4 Calendar & Schedule Dashboard	m04-calendar	modules/m04-calendar/	#09
M5 Leave Management	m05-leave	modules/m05-leave/	#12
M6 Overtime & Shift Scheduling	m06-shifts	modules/m06-overtime-shifts/	#14
M7 Payroll & Payslips	m07-payroll	modules/m07-payroll/	#16
M8 Contributions, Loans & Deductions	m08-contributions	modules/m08-contributions-loans/	#19
M9 Recruitment & Onboarding	m09-recruitment	modules/m09-recruitment/	#21
M10 Talent & Project Contracts	m10-contracts	modules/m10-contracts/	#22
M11 Reports, MIS Dashboard & Insights	m11-reports	modules/m11-reports-dashboard/	#23
M12 Notifications & Self-Service	m12-notifications	modules/m12-notifications-reports/	#25
S1 UI/UX & Database Design	s1-design-db	database/, design/, app/, .github/	#26 (own PRs: PM or a module lead)
S2 QA	s2-qa	tests/	#28
S3 Docs	s3-docs	docs/	#30

Test branch and an example first feature branch

NN is your member number. Feature branch names are the suggestions from Section 6 of the main guide.

Block	Test branch	Example first feature branch (Feature ID)
M1	`docs/m01-hello-NN`	F009: `feature/m01-login-logout`
M2	`docs/m02-hello-NN`	F018: `feature/m02-employee-profile-add-view-edit`
M3	`docs/m03-hello-NN`	F031: `feature/m03-time-time-out-break-logging`
M4	`docs/m04-hello-NN`	F064: `feature/m04-calendar-data-feed-attendance-leav`
M5	`docs/m05-hello-NN`	F051: `feature/m05-leave-types-rules`
M6	`docs/m06-hello-NN`	F058: `feature/m06-shift-templates`
M7	`docs/m07-hello-NN`	F086: `feature/m07-payroll-period-setup`
M8	`docs/m08-hello-NN`	F099: `feature/m08-contribution-tables-sss-philhealth`
M9	`docs/m09-hello-NN`	F105: `feature/m09-job-openings`
M10	`docs/m10-hello-NN`	F109: `feature/m10-contract-records`
M11	`docs/m11-hello-NN`	F113: `feature/m11-hr-dashboard-kpi-cards`
M12	`docs/m12-hello-NN`	F077: `feature/m12-notification-engine-app-email`
S1	`docs/s1-hello-NN`	F001: `feature/s1-erd-data-dictionary`
S2	`test/qa-hello-NN`	F127: `feature/qa-master-test-plan`
S3	`docs/s3-hello-NN`	F140: `feature/docs-project-charter-proposal`

7. Troubleshooting

Problem	What it means	Fix
`Permission denied (publickey)`	GitHub does not know your computer's key	Redo Part 2, Step 3. Run `ssh -T git@github.com` and check the key was added to **your** GitHub account.
Repository not found, or you cannot see your team	Invitation not accepted, or you are not in the team	Accept the invite, then tell your lead or the PM to add you to your team.
Push to staging was refused	The branch is protected	This is correct behavior. Push your own branch and open a PR.
I committed on staging by mistake (not pushed yet)	Your commit is on the wrong branch	`git switch -c feature/m03-my-task`, then `git switch staging`, then `git reset --hard origin/staging`. The commit is safe on the new branch.
The branch name is wrong (not pushed yet)	Typo or missing module code	While on the branch: `git branch -m new-name`. If already pushed, ask your lead.
`! [rejected] (non-fast-forward)`	Someone pushed to your branch before you	`git pull origin <your-branch>`, solve any conflict, then `git push`.
My PR says the branch is out of date	Staging moved on	`git fetch origin`, `git merge origin/staging`, `git push`.
`CONFLICT (content): Merge conflict`	Two people changed the same lines	Fix the markers in the file (Part 4, Step 10). Ask the file's owner if it is not your module.
Your local changes would be overwritten	Unsaved work blocks the switch or pull	`git stash push -m "note"`, do the switch or pull, then `git stash pop`.
Detached HEAD	You are not on a branch	`git switch -c feature/m03-my-task` to keep your work, or `git switch staging` to leave.
The test check fails on my PR	Code or tests broke, or the check itself needs fixing	Open the failed check and read the red lines. Fix, commit, push. If the error is not from your files, tell #26.

Problem	What it means	Fix
No reviewer was added to my PR	The team is empty or CODEOWNERS has a wrong name	Add your lead as reviewer by hand and tell #26.
I added .env or a photo to a commit	A secret or private file is in Git	`git rm --cached .env`, commit, push, tell #26 **immediately**, and change the password or key. It stays in history.

8. For the PM and Technical Lead (#26)

8.1 Pre-flight checklist before the test PRs

- [ ] Every member is in their GitHub team. CODEOWNERS can only request reviewers from teams that have members.
- [ ] **ORG** and the usernames in CODEOWNERS, and in the clone address you send to members, are replaced with real values.
- [ ] Module leads are set as **Maintainers** of their team.
- [ ] Repository settings: default branch is **staging**, Squash merging is allowed, **Automatically delete head branches** is ON.
- [ ] The protection rules for staging and main are saved (see 8.2).
- [ ] The required check named **test** can pass on the current repository contents. The example CI runs `composer install` and `php artisan test`, which need the application code (a composer.json). If the repository only holds the empty folder structure, every test PR will be blocked. Either add the application skeleton first or adjust the workflow.
- [ ] The deploy workflow is added only when the staging server and the three secrets exist (STAGING_HOST, STAGING_USER, STAGING_SSH_KEY).

8.2 Branch protection: decide before relying on it

> **Plan limit**
> On a free plan, branch protection works for public repositories. For a **private** repository it needs a paid plan (Pro or Team). If the rules are not enforced, nothing stops a direct push to staging or main, a merge without approval, a merge with a red check, or a force push.

Option	Effect	Consider
A. Upgrade the organization to a paid plan	All rules in the main guide (Section 2.4) are enforced by GitHub.	Needs billing, which the PM owns. Recommended for a payroll and HR project with 32 members.
B. Stay on the free plan with a private repo	Rules become discipline only. #26 checks merges every day (see below).	Works for a short project if everyone follows Part 3 and 5.5 of the main guide strictly.
C. Make the repository public	Protection works on the free plan.	Not recommended for a client project. Only with the client's explicit approval, and never with real data.

If you stay on option B, #26 checks every day

- Every merged PR had one approval and a green check.
- Nothing was pushed directly to staging or main, and main only received merges from staging.
- No secret or private file (.env, selfies, real payroll data) is in any commit.

```
git fetch origin
git log origin/staging --first-parent --oneline -30
gh pr list --base staging --state merged --limit 30
```

With squash merging, GitHub normally adds the PR number to the commit title, for example *feat(m03): one-time QR generation [F033] (#14)*. A commit on staging without a PR number was probably pushed directly. Ask who did it and why.

> **Check that CODEOWNERS works on a private free repo**
> Automatic reviewer requests from CODEOWNERS may also depend on the plan for private repositories. Confirm with the first test PR. If the lead is not requested automatically, tell every author to add their module lead as reviewer by hand.

If you did upgrade (option A), check that protection works: from a clone with a non-admin test account, run `git push origin staging`. It must be rejected.

8.3 Rollout tracker

Fill in as each team reports. A block is done when all members completed Part 2 and the test PR is merged.

Block	Lead	Members set up	Test PR merged	Checked by / date
M1 Admin, Login & Security	#01	____ of ____	[ ] yes	
M2 Employee Records	#03	____ of ____	[ ] yes	
M3 Attendance	#06	____ of ____	[ ] yes	
M4 Calendar	#09	____ of ____	[ ] yes	
M5 Leave	#12	____ of ____	[ ] yes	
M6 Overtime & Shifts	#14	____ of ____	[ ] yes	
M7 Payroll	#16	____ of ____	[ ] yes	
M8 Contributions & Loans	#19	____ of ____	[ ] yes	
M9 Recruitment	#21	____ of ____	[ ] yes	
M10 Contracts	#22	____ of ____	[ ] yes	
M11 Reports & Dashboard	#23	____ of ____	[ ] yes	
M12 Notifications	#25	____ of ____	[ ] yes	
S1 Design & Database	#26	____ of ____	[ ] yes	
S2 QA	#28	____ of ____	[ ] yes	
S3 Docs	#30	____ of ____	[ ] yes	